

Course Title:	Object-Oriented Software Systems
Course Number:	COE528
Semester/Year:	W2025

Instructor:	Dr. Boujemaa Guermazi
--------------------	-----------------------

<i>Assignment Number/Lab Number:</i>	Project
<i>Assignment/Lab Title:</i>	Project

<i>Submission Date:</i>	30 March 2025
<i>Due Date:</i>	30 March 2025

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Phan	Henry	501232910	18	<i>Henry</i>
Nguyen	Jack	501241070	18	<i>Jack</i>
Gomez Macias	Juan	501243870	18	<i>Juan Gomez</i>

Use-Case (Owner)

1. Name: Add and Manage Books

2. Participating actor: owner.

The owner is responsible for adding, removing, and managing books in the bookstore system.

3. Entry Condition:

- The owner has successfully logged in with the username “admin” and the password “admin”.
- The owner has navigated to the owner-books screen by clicking the [books] button.

4. Flow of Events:

1. The owner enters the book’s name in the “Name” text field.
2. The owner enters the book’s price in the “Price” text field.
3. The owner clicks the [Add] button.
4. The system verifies that both name and price fields are filled.
5. The system adds the new book entry to the books table.
6. The system updates the internal data structure to include the new book.
7. The system confirms the addition of the book by displaying it in the books table.
8. If the owner wants to delete a book, they select a row in the table.
9. The owner selects the [Delete] button.
10. The system removes the selected book from the table and the internal data structure.
11. When finished, the owner clicks the [Back] button to return to the owner-start screen.

5. Exit Condition: The book is successfully added to the system and displayed in the books table.

6. Exceptions:

- If the owner leaves either the name or price field empty when adding a book, the system waits until they do it right.
- If the owner enters a non-numeric value in the price field, the system waits until the owner enters a numerical value in the price field.
- If the owner attempts to delete a book without selecting a row first, the system waits until the owner selects a row.

7. Special Requirements:

- The application ensures that no duplicate book entries exist.
- The GUI provides a clear error message in case of invalid inputs.

- The system automatically updates and saves the book list to “books.txt” when the application exists.
- The table must display book information in two columns: Book Name and Book Price.

Use-Case (Customer)

1. Name: Purchase Books
2. Participating actor: Customer.

The customer can view available books, select books to buy, and complete a purchase using either direct payment or redeemable points.

3. Entry Condition:
 - A registered customer logs into the bookstore application and navigates to the book purchasing screen.
4. Flow of Events:
 1. The customer views the list of available books in the table.
 2. The customer selects one or more books by checking the corresponding checkboxes.
 3. The customer chooses a payment method:
 - Click the [Buy] button to make a purchase using money.
 - Click the [Redeem Points and Buy] button to use accumulated points for a discount.
 4. The system navigates to the customer-cost screen.
 5. The system calculates the total cost of the selected books.
 6. If the customer chooses to redeem points:
 - The system applies all available points to reduce cost(100 points = 1 CAD).
 - The system deducts the redeemed points from the customer’s account.
 7. The system awards new points to the customer based on the final transaction amount (10 points per 1 CAD spent).
 8. The system updates the customer’s status based on points:
 - Silver: less than 1000 points
 - Gold: 1000 points or more
 9. The system displays the final cost, updated point balance, and status.
 10. The system confirms the purchase.
5. Exit Condition:

- The purchase is completed, and the system updates the customer's points and status.
- The customer has logged out of the system, or
- The customer has closed the application, in which case all the customer data is saved to customers.txt.

6. Exceptions:

- If no books are selected when the customer clicks either [Buy] or [Redeem Points and Buy], the system displays a balance of \$0.
- If the customer has insufficient points to cover the entire cost, the system applies all available points and charges the remaining amount.
- If the transaction cost becomes zero or negative after applying points, the final cost is set to zero.

7. Special Requirements:

- The system ensures a secure transaction process.
- The system prevents negative transaction costs.
- The application correctly calculates points (10 points earned per 1 CAD spent).
- The application correctly redeems points (100 points = 1 CAD discount).
- The system correctly transitions between Silver and Gold status based on the 1000-point threshold.
- The GUI displays all transaction details, including total cost, points used, and updated status.
- All customer data must be saved to customers.txt when exiting the application.

State Design Pattern

The State Pattern is a behavioral design pattern that allows an object to change its behavior when its internal state changes. Instead of using if-else or switch statements to check the object's state and execute different code, the State Pattern encapsulates state-specific behavior into separate classes. In the application, the state pattern is applied to manage the customer status (Silver/Gold). Customers start as Silver members and earn points on purchases. Once they acquire enough points, they become Gold members. These Gold members receive discounts and accumulate points at a higher rate. Instead of handling this behavior with multiple conditionals in the customer class, the State pattern is

used to create two separate classes. These two classes, “SilverState” and “GoldState,” each have their own defining behavior for purchases, point accumulation, and possible state transitions.

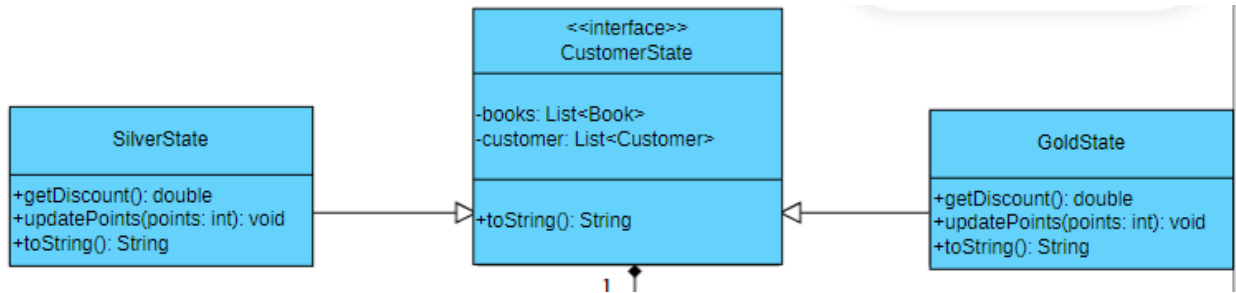


Figure 1: Class Diagram for States.

Using this structure is highly beneficial because it eliminates conditional statements, which can complicate code. Using this structure also encapsulates state-specific behavior within dedicated classes, keeping the customer class clean and focused. This structure also improves the state transitions’ readability by defining them within the state classes. For **SilverState**, there is no discount, and the customer pays full price. However, customers earn 10% of the purchase amount as points. For the **GoldState**, customers receive a 10% discount on purchases, and they also earn 20% of the purchase amount (after discount) as points. Lastly, the state transitions from silver to gold if the customer accumulates 1000 points or more. Anything below 1000 points pertains to the silver state.